Payments and credits

How money works in FunDive. Rules the code depends on; the reasoning behind each one lives in comments at the site that implements it.

FunDive moves no money. There is no payment processor. Bank transfers, cash and card happen off-app and are recorded by hand. Account credit is the one exception — it is internal, and the only "payment" the app creates itself.

Two ledgers

Both append-only. Nothing is ever edited in place: a refund is a new `payments` row, a discount a new `booking_amendments` row.

Table	Holds
`payments`	Money received. `pending` / `paid` / `refunded` / `voided`.
`credits`	Money owed back to a diver. `open` or `settled`.

Every row names the session that wrote it — `recorded_by`, `created_by`, `settled_by`, `cancelled_by` — and the accounting views resolve those to names, so a disputed figure traces to a person rather than to "the app". The trigger-stamped ones (`settled_by`, `cancelled_at`, `cancelled_by`) are derived from the transition and ignore whatever a caller sends, so none can be forged or backdated.

Alongside them, `bookings.details` carries the `total` and `deposit` **frozen at booking time**, so later price changes never alter what someone was quoted.

Every payment names a transaction

A row that moved real money carries a `reference`: a receipt number, a bank transfer id, a PayPal or other online transaction id. Without one, "Paid 3,600" is a claim nobody can check against a bank statement, and a diver holding a receipt has nothing to match it to.

`payments_reference_required` enforces it. Three kinds of row are exempt for the same reason — there is no external transaction to point at: `account_credit` (moves no money; the cash arrived earlier), `pending` (a promise, not a receipt), and `voided` (an admin taking back a mistake).

The constraint is `NOT VALID`: payments predating it keep an honest blank rather than a back-dated fiction, and every write from here on is checked. Divers see the reference on their own payment list — their half of the receipt.

"Mark deposit paid" records no payment at all — it confirms a spot and moves no figure — so it has nothing to reference; the audit log still names who pressed it.

What a booking owes

```
owed       = details.total + Σ booking_amendments.amount
paid       = Σ payments: 'paid' adds, 'refunded' subtracts, rest ignored
credit     = Σ open credits tied to THIS booking
balance    = owed − paid − credit
depositDue = max(0, min(details.deposit, owed) − paid)
```

`netPaid()` and `booking_net_paid()` are the only implementations of that signing rule — every screen and every RPC must use one, or they disagree about a partly refunded booking.

The deposit is **clamped to `owed`**, because a discount can push the frozen deposit above the remaining balance. Covering it promotes a `pending` booking to `confirmed`.

A negative balance is **credit to the diver**, whether it came from an awarded credit or from simply overpaying. There is no separate "overpaid" concept.

Credits

Every credit records a `source`: `manual`, `event_cancellation`, `booking_cancellation_return`, `carry_forward`, or `return_reclaimed`. Only `event_cancellation` and `booking_cancellation_return` (`RETURN_SOURCES`) mean *this booking's money is given back right now*, and only they block a further automatic refund — a goodwill award, a carry-forward remainder or a reclaimed return must not.

Cancelling and restoring is a round trip. The cancel hands the booking's account credit back; restoring takes it back again, because the credit backs the booking once more — settling what is still open, and reducing the booking's account-credit payment by anything already spent elsewhere. Reclaimed rows become `return_reclaimed` so the two triggers agree on state: without that, a second cancel skips refunding *and* a second restore reclaims the same money twice.

Account credit — the balance a diver sees — is *open credits not tied to an active booking, plus every overpayment*. So a credit tied to a cancelled booking is spendable again, and bookings a lead booker pays for count toward the lead, not the diver.

Spending credit

One RPC, `apply_credit_to_booking`, because divers cannot write `payments` or `credits` directly. A diver may spend against their own booking, a parent against their child's, an admin against anyone's — and the credit spent is always the booking owner's.

It clamps to what is owed and what is available, consumes credit oldest-first, carries any remainder forward as a new `carry_forward` row, records an offsetting `account_credit` payment, and confirms the booking if that covers the deposit. It refuses a **cancelled** booking outright: that trip will never happen, so credit spent there would be destroyed.

Cancellations

Four of them, and only one returns money by itself.

What happened	Credited automatically
Shop cancels the event	**Full net paid**, to every registrant
Diver asked to cancel **on or before** the event's cancel-by date	**Full net paid**
Diver asked **after** the cancel-by date	Only the **account credit** they had spent
Admin cancels a booking nobody asked about	Only the **account credit**

Whatever is not credited stays on the booking for a human — a forfeiture never happens automatically.

The deadline is `events.cancel_date`, and the moment that counts is `refund_requested_at` — when the *diver* asked, not when an admin approved, so a slow approval cannot cost them their refund. An event with no cancel-by date has no deadline to miss. `cancellation_in_time()` decides, judging the shop's calendar day via `shop_timezone()` — SQL cannot read `fundive.config.ts`, so a fork restates the zone there and an integration test fails the build if the two disagree.

Account credit is the one thing the app can always hand back on its own, because it never left the app. Cash and transfers did, so only a person can move those.

Cancelling an event does **not** cancel its bookings. The credit is what settles things with the diver. Issuing runs after the cancel commits, so a failure cannot un-cancel the event.

A diver may self-cancel only a `pending` booking with nothing paid and no refund request outstanding (`canSelfCancel`). This is a product rule, not a security boundary — the database lets a diver cancel any of their own bookings — so it must be applied at **every** diver-facing cancel control.

Money left on a cancelled booking

A cancelled booking's balance short-circuits to **settled**. That means *nothing further is owed for this trip*, not *the money has been dealt with*. Its frozen total is not a debt, and netting it would double-count any cancellation credit.

So when a diver who paid cash cancels, the shop still holds their money and no other screen shows it. Three endings are correct, and exactly one must be chosen:

Ending	Recorded as
Money went back	A `refunded` payment
Diver keeps it as credit	An open credit, `booking_cancellation_return`
Shop keeps it (a cancellation fee)	`bookings.cancellation_settled_at`

The first two move money. The third does not — the cash is already counted as revenue on that event — so it records an acknowledgement instead. Without one, a kept fee is indistinguishable from money nobody has dealt with, and its row would never leave the list.

The kept amount is **shown to the diver** on the booking it came off, and to staff on the same booking's payments block. It is derived from what is still on the booking rather than stored, so a later refund cannot contradict it. Settling is staff-only, enforced by `bookings_guard_diver_status` — the self-update RLS policy gates the row, not the columns, so without that a diver could erase their own stranded money from the list.

`/admin/refunds` lists these under **Cancelled bookings still holding money**, with a button per ending. It is the only surface that can see them: balances read settled, account credit skips cancelled bookings, and the refund queue lists only non-cancelled ones.

Reading a diver's balance

The diver profile shows one figure and the history that produced it — a statement, each line carrying the change it made and the balance standing after it (`buildDiverStatement`).

Credit-positive: a positive balance is money the shop owes the diver. The closing balance is deliberately **unclamped**, the one place it differs from `diverCreditBalance` — that function answers "how much can this diver spend", so it floors each active booking at zero. A diver 1,000 short on one dive and 500 ahead on another can spend 500, not −500; a statement cannot floor anything or its lines stop adding up. Both are shown, and they agree unless some active booking is underpaid.

Three rules make it reconcile. A cancelled booking's charge **and payments** are reversed at `cancelled_at`, and anything recorded against it afterwards has no effect; a credit tied to a cancelled booking still counts, because that is how a refund-as-credit reaches the diver; and a booking a lead booker pays for is dropped, being the lead's money. Which leaves the identity the summary block is built on: `balance = paid − charged + open credit`.

Reporting

Applied account credit is **not revenue**. The cash arrived earlier, on the booking that generated the credit; counting it again books the same money twice. `isExternalPayment()` guards every cash-revenue sum, and credit spent is reported beside cash, never inside it.

Balance math is the opposite and counts it in full — it really did settle that booking. Staff revenue is different again: base price × confirmed heads, deliberately not what has been banked.

Rules that must not break

1. Never edit money rows in place. New row, always.
2. Sum paid through `netPaid` / `booking_net_paid`. Nowhere else.
3. `owed` is `details.total` **plus amendments**, in TypeScript and SQL.
4. Clamp the deposit to `owed` wherever you compare against it.
5. A booking's own credit is never spendable against that booking.
6. A cancelled booking has no live balance and refuses credit.
7. `diverCreditBalance` is the only definition of account credit.
8. Every automatic credit records its `source`.
9. Cash-revenue sums apply `isExternalPayment`. Balance math does not.
10. A money-moving payment names its real-world transaction, and attribution is stamped from the act, never taken from the caller.
11. Constraints, triggers and RLS policies get integration tests.